



Wholly owned by UTAR Education Foundation
(Co. No. 578227-M)
DU012(A)

UNIVERSITI TUNKU ABDUL RAHMAN

**FACULTY OF INFORMATION AND COMMUNICATION
TECHNOLOGY**

UCCD2063 ARTIFICIAL INTELLIGENCE TECHNIQUES

GROUP ASSIGNMENT

YEAR AND SEMESTER: 2024/06

Student Name:	Lim Wei Haw	Lau Kai Lin	Wong Pei Kei
Student ID:	2207307	2204742	2207466
Contribution:	33.33%	33.33%	33.33%
Signature			

Table of Contents

Content	Page Number
Chapter 1: Introduction - 1.1 Background - 1.2 Objective	1
Chapter 2: Methods - 2.1 Dataset Description - 2.2 Data Exploration and Visualization - 2.3 Data Preprocessing - 2.4 Model Selection - 2.5 Model Training and Validation - 2.6 Model Tuning and Testing	2 - 24
Chapter 3: Result and Discussion - 3.1 Training and Testing Result - 3.2 In-Depth Analysis of Prediction Performance - 3.3 Strengths and Weaknesses of Models - 3.4 Features Importance	25 - 35
Chapter 4: Conclusion	36

Chapter 1: Introduction

1.1 Background

In this era, cardiovascular (CVD) diseases are serious global health problem which causing millions of deaths every year. These diseases are often preventable, but their complexity and various influencing factors make early detection and effective risk assessment become challenging. Traditional methods of assessing cardiovascular risk typically involve analyzing individual clinical measures such as blood pressure, cholesterol levels and lifestyle factors. Although these methods provide valuable insights, they may not fully capture the complex interactions among variables that contribute to cardiovascular risk.

The problem at hand is the need for a more accurate and comprehensive method to predict cardiovascular risk levels. In order to achieve this, using machine learning techniques is a effective way. Machine learning algorithms can analyze large data sets containing different variables, identify complex patterns and make predictions that may not be obvious with traditional analysis methods. Through these capabilities, predictive models can be developed to more accurately assess an individual's cardiovascular risk, thereby aiding in early diagnosis and prevention strategies.

1.2 Objective

The primary objective of this project is to let student learn how to develop and train machine learning models that can accurately predict an individual's cardiovascular risk level (low, medium, or high) using a provided dataset. In order to achieve this, the following steps will be perform:

1. Data Exploration: Explore the dataset understand the distribution and relationships between features, detect any patterns and identify correlations important for predicting cardiovascular risk.
2. Data Prerocessing: Clean and prepare datasets for analysis, divided the data into a training set and a test set to evaluate the performance of the model.
3. Model Selection and Training: Evaluate various machine learning algorithms to determine the best model for predicting cardiovascular risk.
4. Model Tuning: Fine-tune the selected models by adjusting the hyperparameter to improve prediction accuracy

Chapter 2: Methods

2.1 Dataset Description

The figure 2.1 shows that the dataset “dataset.csv” consists of 2100 samples which include 8 numerical features and 10 categorical features and require memory around 295.4KB to load it. The numerical features include age, height(cm), weight(kg), vege_day, meals_day, water_intake(L), exercise and income. The categorical features include gender, family_history, alcohol, junk_food, snack, smoking, transportation, tv, discipline and cardiovascular_risk(y). From the dataset, features #0 to #16 serve as the input features and #17 is the output features. The output feature corresponds to the target variable which categorizes the cardiovascular risk level for each individual.

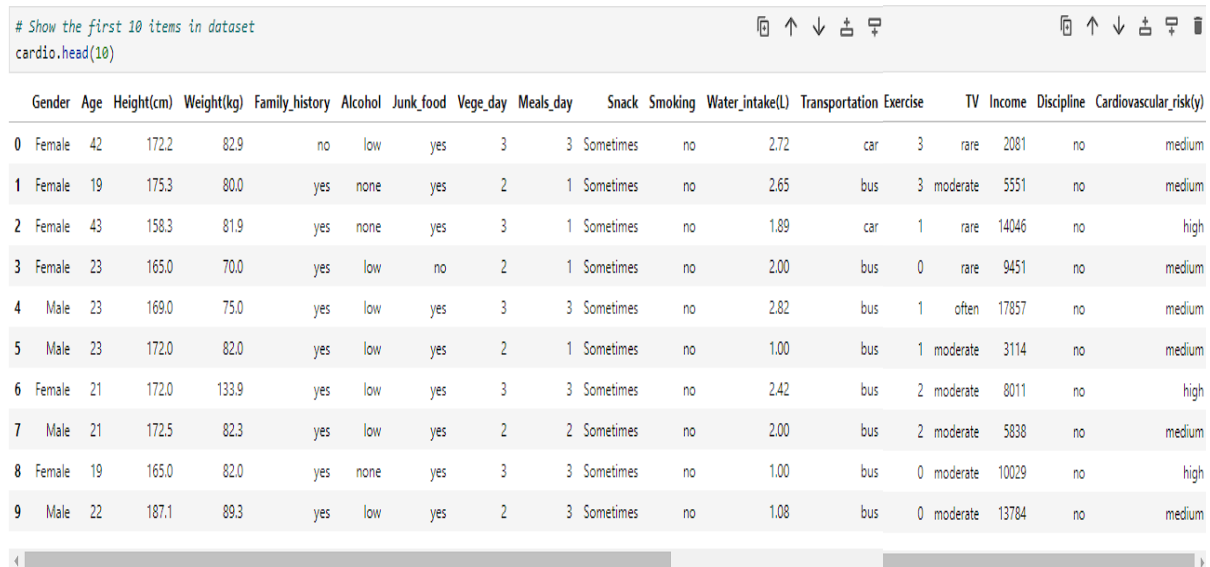
```
# Show the quick description all attributes in the DataFrame cardio
cardio.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2100 entries, 0 to 2099
Data columns (total 18 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   Gender                2100 non-null  object  
 1   Age                  2100 non-null  int64   
 2   Height(cm)           2100 non-null  float64  
 3   Weight(kg)            2100 non-null  float64  
 4   Family_history        2100 non-null  object  
 5   Alcohol              2100 non-null  object  
 6   Junk_food            2100 non-null  object  
 7   Vege_day             2100 non-null  int64   
 8   Meals_day            2100 non-null  int64   
 9   Snack                2100 non-null  object  
10   Smoking              2100 non-null  object  
11   Water_intake(L)      2100 non-null  float64  
12   Transportation        2100 non-null  object  
13   Exercise              2100 non-null  int64   
14   TV                   2100 non-null  object  
15   Income               2100 non-null  int64   
16   Discipline            2100 non-null  object  
17   Cardiovascular_risk(y) 2100 non-null  object  
dtypes: float64(3), int64(5), object(10)
memory usage: 295.4+ KB
```

Figure 2.1: Dataset Information

2.2 Data Exploration and Visualization

First, we can observe the first 10 samples in the dataset to get a quick overview of the data structure and format including the types of features and their values. We can identify any obvious data such as missing values, incorrect data types or anomalies. This helps catch errors early in the data preparation process. Furthermore, we can identify there is how many features including numerical and categorical features.



	Gender	Age	Height(cm)	Weight(kg)	Family_history	Alcohol	Junk_food	Vege_day	Meals_day	Snack	Smoking	Water_intake(L)	Transportation	Exercise	TV	Income	Discipline	Cardiovascular_risk(y)
0	Female	42	172.2	82.9	no	low	yes	3	3	Sometimes	no	2.72	car	3	rare	2081	no	medium
1	Female	19	175.3	80.0	yes	none	yes	2	1	Sometimes	no	2.65	bus	3	moderate	5551	no	medium
2	Female	43	158.3	81.9	yes	none	yes	3	1	Sometimes	no	1.89	car	1	rare	14046	no	high
3	Female	23	165.0	70.0	yes	low	no	2	1	Sometimes	no	2.00	bus	0	rare	9451	no	medium
4	Male	23	169.0	75.0	yes	low	yes	3	3	Sometimes	no	2.82	bus	1	often	17857	no	medium
5	Male	23	172.0	82.0	yes	low	yes	2	1	Sometimes	no	1.00	bus	1	moderate	3114	no	medium
6	Female	21	172.0	133.9	yes	low	yes	3	3	Sometimes	no	2.42	bus	2	moderate	8011	no	high
7	Male	21	172.5	82.3	yes	low	yes	2	2	Sometimes	no	2.00	bus	2	moderate	5838	no	medium
8	Female	19	165.0	82.0	yes	none	yes	3	3	Sometimes	no	1.00	bus	0	moderate	10029	no	high
9	Male	22	187.1	89.3	yes	low	yes	2	3	Sometimes	no	1.08	bus	0	moderate	13784	no	medium

Figure 2.2: First 10 Sample in Dataset

It is necessary to check whether there is missing value in each features so that the prediction can be done accurately. Figure 3 below shows the NULL value of each features. From the figure 2.3, we observed that all the features are filled with appropriate value.

```
# Check missing value of all features
cardio.isnull().sum()

Gender                0
Age                  0
Height(cm)           0
Weight(kg)           0
Family_history       0
Alcohol              0
Junk_food            0
Vege_day             0
Meals_day            0
Snack                0
Smoking              0
Water_intake(L)      0
Transportation       0
Exercise             0
TV                   0
Income               0
Discipline           0
Cardiovascular_risk(y) 0
dtype: int64
```

Figure 2.3: Determine Missing Value in Each Features

Moreover, examine attributes in categorical features is important. It provides a clear understanding of the different categories present in each feature which is important for interpreting data and make decisions about preprocessing. From the figure 2.3 below, we can observed all the attributes of each categorical feature.

```
# Identifying categorical features by checking data types
cat_features = cardio.select_dtypes(include=['object', 'category']).columns

# List the exact attributes for each categorical feature
for feature in cat_features:
    print(f"{feature}: {cardio[feature].unique()}")

Gender: ['Female' 'Male']
Family_history: ['no' 'yes']
Alcohol: ['low' 'none' 'medium' 'high']
Junk_food: ['yes' 'no']
Snack: ['Sometimes' 'Always' 'Frequently' 'No']
Smoking: ['no' 'yes']
Transportation: ['car' 'bus' 'walk' 'motorcycle' 'bicycle']
TV: ['rare' 'moderate' 'often']
Discipline: ['no' 'yes']
Cardiovascular_risk(y): ['medium' 'high' 'low']
```

Figure 2.3: Attributes of Each Categorical Features

Figure 2.4 below shows the table that provides summary statistics for the numerical features in the dataset. The summary statistics shown the count which is the number of non-missing values in each column. All columns have 2100 entries represent that there are no missing values in these features. Mean is the average value of each feature. Std (Standard Deviation) is a measure of the variability or dispersion of the data around the mean. Min (Minimum) represents the smallest value in each column. 25% represent the value below which 25% of the data falls. For example, 25% of the individuals are below 20 years old and 25% of the individuals are 163cm in height. That is same for 50% and 75% respectively. Max (Maximum) is the largest value in each column. Furthermore, the statistics show the range of values for each feature. For example, by taking the maximum value and subtracting the minimum value, we can know that the age range of individuals in this dataset is between 14 and 61 years old.

```
#Show all the numerical attributes in cardio
cardio.describe()
```

	Age	Height(cm)	Weight(kg)	Vege_day	Meals_day	Water_intake(L)	Exercise	Income
count	2100.000000	2100.000000	2100.000000	2100.000000	2100.000000	2100.000000	2100.000000	2100.000000
mean	24.302381	170.148286	86.561571	2.423333	2.687143	2.007429	1.006190	9432.386190
std	6.342270	9.340941	26.192242	0.584318	0.810088	0.613122	0.894885	5002.350673
min	14.000000	145.000000	39.000000	1.000000	1.000000	1.000000	0.000000	1000.000000
25%	20.000000	163.000000	65.400000	2.000000	3.000000	1.577500	0.000000	4994.500000
50%	23.000000	170.000000	83.000000	2.000000	3.000000	2.000000	1.000000	9226.500000
75%	26.000000	176.800000	107.250000	3.000000	3.000000	2.480000	2.000000	13841.750000
max	61.000000	198.000000	173.000000	3.000000	4.000000	3.000000	3.000000	18000.000000

Figure 2.4: Summary Statistics of Numerical Features

The figure 2.5 below shows the histograms of each numerical features in the dataset. The histogram of age shows a right-skewed distribution which most of the individuals are between 18 and 30 years old. The height data follows a normal distribution which most individuals having a height between 160cm and 180cm. The weight distribution is fairly spread out with a higher concentration between 70kg and 90kg. The histogram of vegetable servings shows a highly discrete distribution with most individual has 2 or 3 servings of vegetable per day. The histogram of meal servings also shows a discrete distribution with most individual has 3 servings of meal per day. The histogram of water intake shows a similar discrete distribution with most individual has served 2 litres of water per day. The exercise data shows that most individual exercise 0 to 2 times per day. Finally, the income distribution is fairly uniform and there is no strong skew in the distribution.

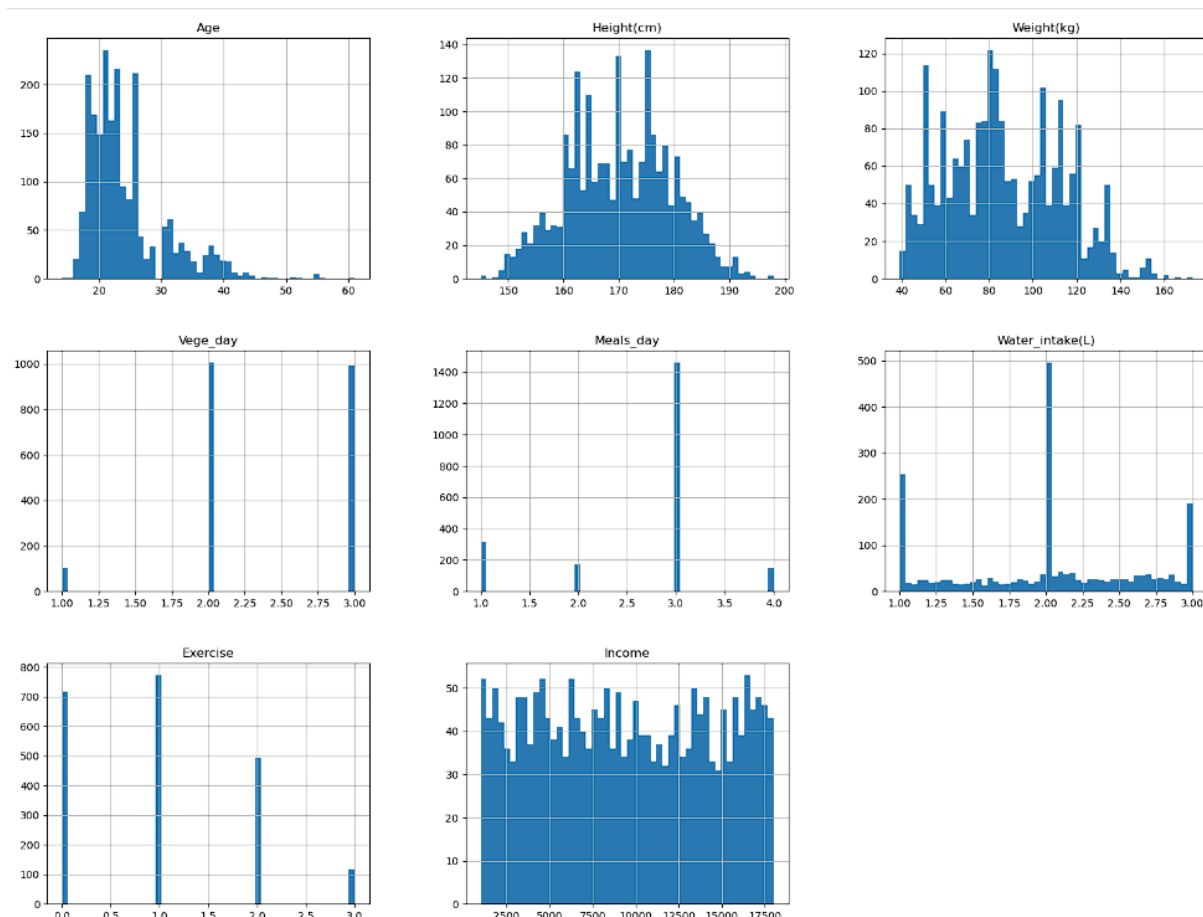


Figure 2.5: Histogram of Numerical Features

Figure 2.6 show the box plot that represents the distribution of ages across three cardiovascular risk. The box plot helps us to understand the median age, the spread (interquartile range or IQR) and the presence of outliers in each risk group.

In this visualization, the median age of individuals with lower cardiovascular risk is relatively low with most individuals appearing to be in their early 20s. The interquartile range (height of the box) for this group is relatively small which indicating that most individuals in the low-risk category fall into a narrow age range. Despite this, there are a few outliers represented by the dots above the beards who are older than most of the group. These outliers suggest that while cardiovascular risk is generally lower in younger adults, older adults may still be classified as low risk in special circumstances. This may be due to various other protective factors such as lifestyle, genetic predisposition or good health maintenance.

The median age was slightly higher in the moderate-risk group compared with the low-risk group, this shows that individuals in this group were older on average. The age distribution of this group was also broader, indicating greater age variability among individuals classified as moderate cardiovascular risk. Similar to the low-risk group, there are outliers, but in this case the outliers represent both younger and older adults, this shows that different age groups fall into this category based on other risk factors.

The high-risk group interestingly has a lower median age than the medium-risk group although not by a large margin. This shows that while age is a factor, younger individuals may still face higher cardiovascular risks, this possibly due to other contributing factors such as poor diet, lack of exercise, obesity or genetic predispositions. The outliers in this group also reveal that some older individuals are classified as high risk, consistent with the idea that advancing age typically increases cardiovascular risk although not universally.

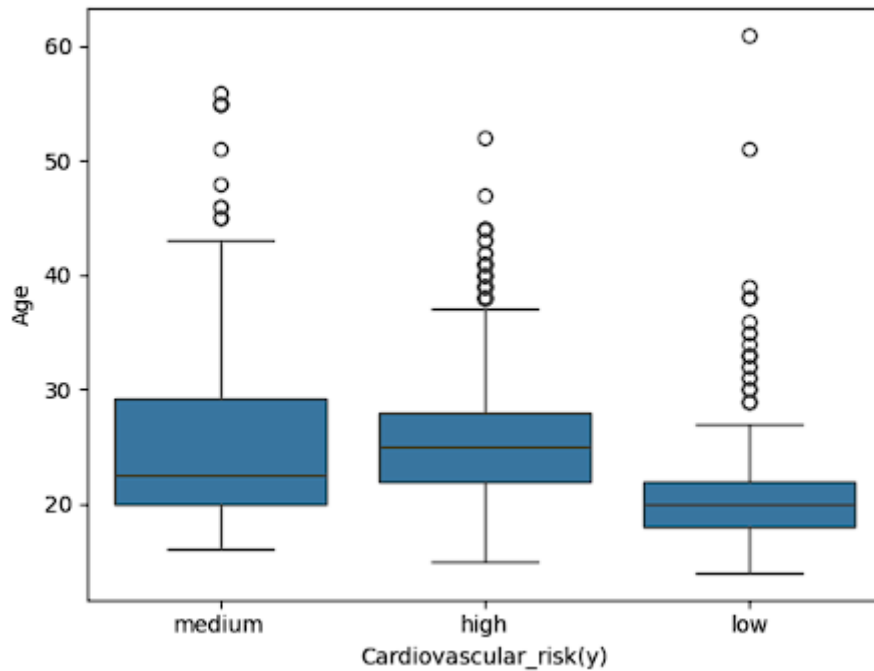


Figure 2.6: Box Plot of Age vs Cardiovascular Risk

Figure 2.7 shows the heatmap that visualizes the correlation matrix of various variables related to cardiovascular risk and factor. Each cell in the heatmap represents the correlation coefficient between two variables, with values ranging from -1 to 1. Age shows a moderate positive correlation with Weight (0.20) which as age increases, weight tends to increase as well. Other correlations are quite weak, close to 0. Height is moderately correlated with Weight (0.46), meaning that taller individuals tend to weigh more. Weight shows a moderate positive correlation with Age (0.20), Height (0.46) and Water intake (0.20). This means that individuals who weigh more also tend to drink more water. Vegetable intake shows a weak positive correlation with Weight (0.18) and very weak correlations with other variables. Meals per day have a weak positive correlation with Height (0.24) and very weak correlations with other variables. Water intake shows a weak positive correlation with Weight (0.20) and Exercise (0.14). Exercise has a weak positive correlation with Height (0.29) and Water intake (0.14) which shows that taller individuals and those who exercise more tend to drink more water. Last but not least, income has very weak correlations with all other variables. Overall, the heatmap shows that the most significant correlation is between height and weight (0.46).

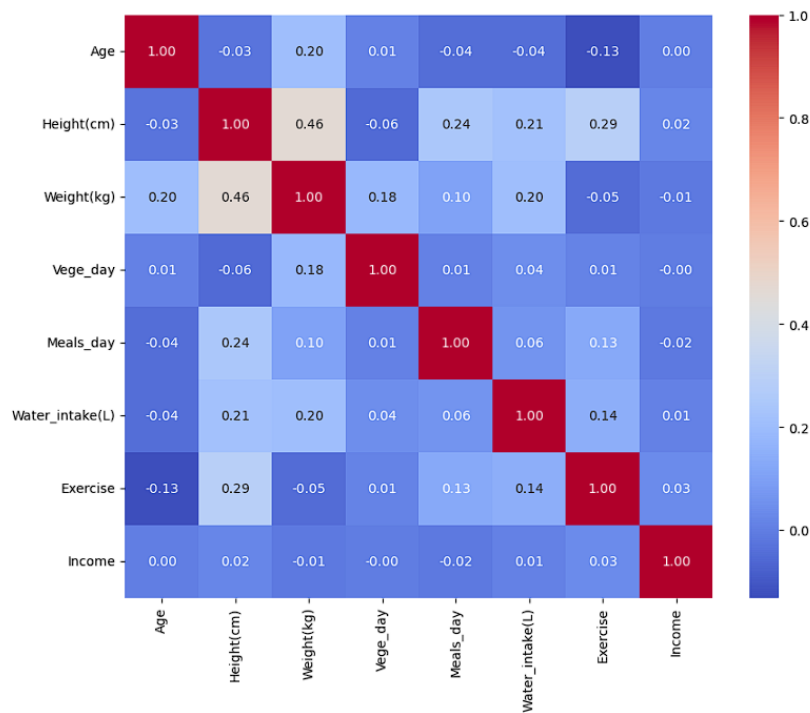


Figure 2.7: Correlation Matrix of Different Factors

Figure 2.8 shows the bar chart that illustrates the relationship between smoking status and cardiovascular risk. The x-axis represents smoking status, with two categories: "no" for non-smokers and "yes" for smokers. The y-axis shows the count of individuals within each risk category. The chart shows that the majority of people in the dataset are non-smokers but with a large proportion classified as having high cardiovascular risk. Although there are fewer smokers, they are still present in all risk categories with a slight predominance in the high-risk category. These data may mean that although smoking is associated with cardiovascular risk, nonsmokers also show a higher prevalence of cardiovascular risk and this is possibly due to other factors not listed in this chart.

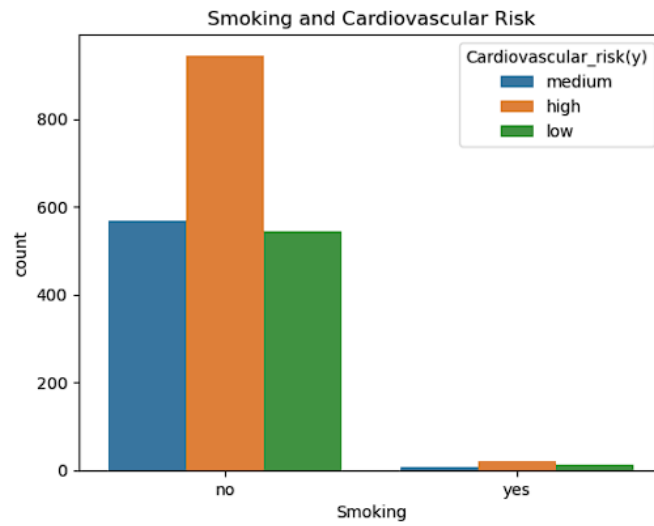


Figure 2.8: Relationship between Smoking and Cardiovascular Risk

2.3 Data Pre-processing

Data pre-processing is a important step in preparing the dataset for modeling. This stage involves split the dataset into training and testing set, scaling numerical data and encoding categorical data to ensure the model can accurately laern from the data.

The first step in data preprocessing is split the dataset into training and testing set. Before we split the training and testing set, we first separate the output vector(y) from the input matrix(X). In this case, output vector(y) will be the Cardiovascular_risk(y) column which indicates the risk level of cardiovascular disease. After we drop the cardiovascular_risk(y) column, input matrix(X) only consists of 16 features other than cardiovascular_risk(y).

Next, we divided the dataset into training and testing sets using 'train_test_split' with 80% of the data reserved for training and 20% for testing. This partitioning is critical for evaluating a model's performance on unseen data which ensuring that the model not only remembers the training data but also generalizes well to new data. The figure below shows the shape of training(80% of 2100 = 1680) and testing (20% of 2100 = 420) set after splitting.

```
X = cardio.drop('Cardiovascular_risk(y)', axis=1) # Drop the target column to get features
y = cardio['Cardiovascular_risk(y)'] # Target variable

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=30)
print ('full set shape=', X.shape)
print ('train shape=', X_train.shape)
print ('test shape=', X_test.shape)

full set shape= (2100, 17)
train shape= (1680, 17)
test shape= (420, 17)
```

Figure 2.9: Shape of Train, Test and Full Set

After that, we separated the input matrix into numerical and categorical sets. This step is important because numerical and categorical data requires different preprocessing techniques. We first identify the numerical columns and categorical columns by select data types of each column and split into corresponding variables. Then, we split it into numerical and categorical training sets.

```

# Identify the numerical columns and categorical columns
num_cols = X.select_dtypes(include=['int64', 'float64']).columns
cat_cols = X.select_dtypes(include=['object']).columns

# Split the input matrix into numerical and categorical sets
X_train_num = X_train[num_cols]
X_train_cat = X_train[cat_cols]

print('X_train_num shape:', X_train_num.shape)
print('X_train_cat shape:', X_train_cat.shape)

X_train_num shape: (1680, 8)
X_train_cat shape: (1680, 9)

```

Figure 2.10: Shape of Train Number and Train Categorical Set

The next important step is preprocessing both numerical and categorical data. For numerical data, we applied ‘StandardScaler’ for the standardization. This is to calculate the mean and standard deviation for each numerical feature based on the training set. We first create the scaler object called scaler. Then we fit in the training set X_train_num into scaler object to compute the statistics which later used to standardize both the training and test data. The numerical features in the training set are then transformed using ‘scaler.fit_transform’. This scales the data so that each feature has a mean of zero and a standardization of one. The same transformation is applied to the test data to ensure consistency in scaling across both datasets.

```

# Preprocessing numerical data: Standardization
scaler = StandardScaler(copy = False)
scaler.fit(X_train_num)
X_train_num = scaler.transform(X_train_num)
X_test_num = scaler.transform(X_test[num_cols])
print('X_train_num shape:', X_train_num.shape)
print('X_test_num shape:', X_test_num.shape)
print('>>> Mean of all columns:\n', X_train_num.mean(axis=0))
print('\n>>> Standardization of all columns:\n', X_train_num.std(axis=0))

X_train_num shape: (1680, 8)
X_test_num shape: (420, 8)
>>> Mean of all columns:
[-1.98782789e-16  1.10440757e-15  5.64627710e-16 -1.34284118e-16
 -7.19001578e-17  3.21436000e-16  1.01506105e-16 -1.52259158e-16]

>>> Standardization of all columns:
[1. 1. 1. 1. 1. 1. 1. 1.]

```

Figure 2.11: Standardization

For categorical data, since we have multiple categorical features, we applied ‘OneHotEncoder’ to encode it. We create the OneHotEncoder object called oh_encoder. We initialized the ‘OneHotEncoder’ with the parameters ‘sparse_output = False’. The ‘sparse_output = False’ ensures that the output was a dense array which is easier to work with in subsequent steps. The encoder was fitted to the categorical training data. Once fitted, categorical features are converted into one-hot encoded arrays where each unique category in the feature is converted into a separate binary column. This process works for both training and test sets.

```
# Preprocessing categorical Data: One-hot encoding
oh_encoder = OneHotEncoder(sparse_output=False)
oh_encoder.fit(X_train_cat)
X_train_cat = oh_encoder.transform(X_train_cat)      # returns numpy array
X_test_cat = oh_encoder.transform(X_test[cat_cols])
print('Shape of X_train_cat:', X_train_cat.shape)
print('Shape of X_test_cat:', X_test_cat.shape)

Shape of X_train_cat: (1680, 26)
Shape of X_test_cat: (420, 26)
```

Figure 2.12: One-Hot Encoder

After done data preprocessing, we have to finalize the training and test set by combining the numerical and categorical feratures into a single training array ‘X_train_tr’ and a single array ‘X_test_tr’. This is done by using Numpy’s function (hstack) which horizontally stacks the numerical and categorical feature arrays. After this step, ‘X_train_cat’ and ‘X_test_cat’ both have 34 columns which correspond to the combined set of numerical and categorical features. Finally, we convert ‘y_train’ and ‘y_test’ to numpy array format for subsequence model training process. ‘y_train’ consists the label ‘Cardiovascular_risk(y)’ for the training data while ‘y_test’ consists the same label for testing data.


```
# Finalizing the training set
X_train_tr = np.hstack([X_train_num, X_train_cat])
X_test_tr = np.hstack([X_test_num, X_test_cat])

# Convert y_train and y_test to numpy arrays
y_train = y_train.values
y_test = y_test.values

print('X_train_tr shape:', X_train_tr.shape)
print('X_test_tr shape:', X_test_tr.shape)
print('y_train shape:', y_train.shape)
print('y_test shape:', y_test.shape)

X_train_tr shape: (1680, 34)
X_test_tr shape: (420, 34)
y_train shape: (1680,)
y_test shape: (420,)
```

Figure 2.13: Shape of After Preprocessing Train and Test Set

2.4 Model Selection

Here we choose four different models: the K-Nearest Neighbors, Support Vector Classification, Logistic Regression, and Random Forest, which will be trained for the prediction of cardiovascular risk. Each of these models possesses certain strengths that make them useful in various respects on our dataset.

K-Nearest Neighbors (KNN) is a very simple, non-parametric algorithm that performs quite well on datasets where the decision boundaries are non-linear. More specifically, it simply assigns each data point to its most frequent class among the k-nearest neighbors in feature space. It does so by calculating the distance between the points in the data and then classifies new points based on the majority class of their nearest neighbors using that calculated distance.

Support Vector Classification (SVC) is versatile for both linear and non-linear classification problems with different kernels. It seeks the best hyperplane in feature space, which maximizes the margin between classes. Employing kernel functions of various classes-a line, a polynomial, a radial basis function, among others-SVC can accommodate complex data distributions.

Logistic Regression turns out to be a good baseline model simply for its simplicity and interpretability, especially on binary outcomes. In Logistic Regression, the probability of class outcome is modeled using a logistic function. It estimates the relationship between the dependent binary variable and one or more independent variables by fitting a logistic curve. This can also be extended to multi-class problems using techniques such as one-vs-rest.

Random forest does not easily fall into overfitting, handles high-dimensional data well, and it deals well with missing values. This is an ensemble method that constructs many decision trees during training. Each one is built based on a random subset of the data, and the final prediction gets made by aggregating the predictions of all trees by majority voting-for classification-or averaging-for regression. This reduces variance and, as such, improves model accuracy.

2.5 Model Training and Validation

K-Nearest Neighbors (K-NN)

After performed data preprocessing, we can start to train the K-NN model. We first create the k-nn object called model and initialized it with a specified number of neighbors which is 5 in our case. This means the model consider the 5 closest neighbors to make a prediction. The K-NN model was trained on the combined feature matrix 'X_train_tr' and the target variable 'y_train' by using 'fit' method. After that, we start to generate the predictions on the training set using the trained model. Furthermore, accuracy was calculated to evaluate the model's performance on the training set. Before resampling on train set, the training accuracy was found to be 89.88% which shown that the proportion of correctly predicted instances to the total number of training samples. Additionally, cross-validation was employed to evaluate the model's performance across different subsets of the training data. This help evaluate how well the model generalizes to unseen data and reduces the risk of overfitting. In our case, we computes accuracy scores for 5 different folds. The accuracy scores for each fold were 0.866, 0.869, 0.824, 0.818, 0.798 and the mean accuracy among all folds was 83.51%. This provides a more generalized performance metric which reflects the model's ability to perform well across different data splits. Nevertheless, the confusion matrix obtained from cross-validation shows the distribution of prediction versus the actual labels. This helps in understanding the model's performance on different classes and identifying potential areas for improvement. After perform resampling technique, the training accuracy found to be 94.92%, the mean of cross validation among five folds were 90.91%. Overall, these results show that before resampling, the mdoe had good training accuracy but struggled to generalize effectively as indicated by the lower cross validation scores. After applying resampling, both training accuracy and cross validation accuracy improved. This show that the model now performs better on both the training data and unseen data with less risk of overfitting.

Support Vector Classifier

We import the model SVC from the library `sklearn.svm` and instantiate the class with the default settings: `random_state=42`; `probability=True` to receive probability estimates; `class_weight='balanced'`, just in case the classes are imbalanced.

Next, we fit the model using our data for training features, that is, `'X_train_tr'` and training labels, that is, `'y_train'`. This is followed by the evaluation of model performance on training data. The accuracy on the training data set is 98.45%. We will generate a classification report. The results from that are as follows: precision-macro is 0.9816, recall-macro is 0.9831, F1 score-macro is 0.9823, precision-weighted is 0.9848, recall-weighted is 0.9845, and F1 score-weighted is 0.9846. Also, plot the confusion matrix representing the true positives, false positives, true negatives, and false negatives.

Then, we assess the performance of the model by cross-validation. After several rounds of cross-validation, some accuracy scores are represented as: [0.9554, 0.9464, 0.9494, 0.9435, 0.9494], and the average accuracy is 94.88%. Now, we plot the precision-recall curve and ROC curve, and calculate their AUC values so that we can see the performance of the model in distinguishing between classes.

We also plot learning curves to show how the model's performance changes as we change the amount of training data. After using SMOTE to balance the classes in the training data, we retrain the SVC model on the resampled dataset. The training accuracy has now increased to 99.27%. Using resampled data, we repeat cross-validation and obtain accuracy scores of [0.9638, 0.9680, 0.9595, 0.9850, 0.9936], with a mean accuracy of 97.40% correspondence.

Logistic Regression

We import the Logistic Regression model from the "sklearn.linear_model" package and start training it. In order for the results of this problem to be reproducible, we set `multi_class='multinomial'`, `solver='lbfgs'`, `max_iter=1000`, and `random_state= 42`.

Now we will train the model on the training dataset comprising labels "y_train" and features "X_train_tr.". Model performance is measured over the set of training data. From the above, it can be observed that 98.51% of the total cases in the training set were correctly classified. Now, we create the classification report with the following macro values: F1 score: 0.9821, recall: 0.9821 and precision: 0.9826. The weighted metrics are 0.9855 for precision, 0.9851 for recall, and 0.9851 for F1 score. We plot the confusion matrix to visualize the performance.

Further, the cross-validation performance of the model is evaluated. The cross-validation accuracy scores with an average of 97.26% is given by [0.9821, 0.9583, 0.9702, 0.9821, 0.9702]. In the cross-validated performance, the respective precision, recall, and F1 score for the macro metric were 0.9684, 0.9674, and 0.9676, while for weighted metrics, they were 0.9732, 0.9726, and 0.9727, respectively. To check the model performance, we plot learning curve, ROC curve, and precision-recall curve. Then, we run our model on the test set from which we receive an accuracy of 97.38%.

We again do logistic regression on the resampled data after its resampling via SMOTE. Its accuracy rises to 98.85% during the training. We also achieve through cross-validation on resampled data accuracy values of [0.9723, 0.9787, 0.9723, 0.9893, 0.9957] with mean accuracy of 98.17%. The cross-validated performance after SMOTE gives accuracy of 98.16%, precision (macro) of 0.9822, recall (macro) of 0.9816, and F1 score (macro) of 0.9817. Weighted metrics include Precision: 0.9822, Recall: 0.9816, and F1: 0.9817.

Random Forest

Importing `RandomForestClassifier` from the `sklearn.ensemble` package, we first build an instance of the Random Forest model with `random_state=42` to ensure that each run yields precisely the same result and `n_estimators=100` to employ 100 trees in the forest. This allows us to start training the Random Forest model.

Subsequently, we assess the model's performance on the training data after training it on the training dataset (`X_train_tr` for features and `y_train` for labels). With precision (macro), recall (macro), F1 score (macro), weighted precision, weighted recall, and weighted F1 score of 1.0000, the Random Forest model yields 100% accuracy on the training data. Lastly, we plot the confusion matrix to gain a sense of the model's flawless performance on the training set.

Using cross-validation, we further validate the model's performance. With an average accuracy of 95.65%, the cross-validation accuracy scores are [0.9554, 0.9464, 0.9613, 0.9524, 0.9673]. Recall (macro) is 0.9501, F1 score (macro) is 0.9496, and precision (macro) is 0.9491 based on the cross-validated performance metrics. The precision, recall, and F1 score are weighted at 0.9567, 0.9565, and 0.9566, respectively. To further evaluate the model, we additionally plot learning curves, ROC, and precision-recall curves. Lastly, a 97.38% accuracy test is performed on the test set to evaluate the model.

The Random Forest model was trained once more using the resampled data, `X_train_res` and `y_train_res`, after the use of SMOTE to balance the classes in the training data. Once again, the model performs 100% accurately during training. It has provided the following accuracy scores during cross-validation: [0.9680, 0.9616, 0.9616, 0.9893, 0.9893]; the average accuracy is 97.40%. Accuracy of 97.40%, precision (macro) of 0.9741, recall (macro) of 0.9740, and F1 score (macro) of 0.9740 are the cross-validated results for SMOTE. Recall was 0.9740, F1 score was 0.9740, and weighted precision was 0.9741.

2.6 Model tuning and testing

K-Nearest Neighbors (K-NN)

In order to optimize the K-NN algorithm, hyperparameter tuning was performed using an exhaustive grid search technique. This method involves evaluating every possible combination of hyperparameter values to identify the optimal settings for the model. In this case, we tuned three key hyperparameters which is `n_neighbors`, `weights` and `metric`.

Selecting the optimal `k` value involves balancing the trade-off between bias and variance. A smaller `k` value can lead to a model that is over sensitive to noise (high variance) while a larger `k` value might make it too general (high bias). In order to find the best `k` value, We explored a range of values including 7, 9, 11, 13, 15, 17, 19 and 21. The choice of `k` significantly impacts the model's performance, as a smaller `k` can make the model sensitive to noise, while a larger `k` can make it too general.

For the hyperparameter '`weights`', this defines how the influence of each neighbor is weighted when making predictions. In our case, we tested two options which is '`uniform`' and '`distance`'. '`Uniform`' represents all neighbors have the qual weight while '`distance`' represents neighbors are weighted inversely by their distance from the query point which means the closer neighbours will have a higher influence on the prediction.

For the hyperparameter '`metric`', we evaluated three metrics which is '`Euclidean`', '`Manhattan`' and '`Minkowski`'. '`Euclidean`' represents the straight-line distance between two points in Eculidean space. '`Manhattan`' is the sum of the absolute differences between the coordinates of the points while '`Minkowski`' is a generalized distance metric that includes both Euclidean and Manhattan distances as special cases depending on the parameter `p`.

For instance, we utilized a 5-fold cross-validation approach to evaluate the performance of each combination of hyperparameters. This technique splits the training data into five subsets, training the model on four of these subsets and validating on the remaining one. This process repeated five times with each subset serving as the validation set once. The mean accuracy score across all folds is used to evaluate the performance of each combination.

Based on the result of grid search, the optimal hyperparameters for the K-NN model are `k` value 9 for the number of neighbors, '`distance`' for `weights` and '`Manhattan`' for `metric`. The test accuracy increased from 89.76% to 90.95% after fine-tuning which show the improvement in the model's ability to generalize to unseen data.

Support Vector Classifier

We performed a grid search in order to tune the SVC for the best performance by choosing different hyperparameter values: C, type of kernel, gamma, and degree of polynomial. The search is over in order to find an optimum combination of these parameters that will make this model most accurate. The regularization parameter, C, controls the trade-off between low training error and a simple model. We have tried values of 0.1, 1, 10, and 100. We used several types of kernels by trying 'linear', 'rbf', and 'poly' in order to check which type of transformation is better for the dataset, such that good classification can be done. For Gamma, we tried values between 0.001 to 1, including 'scale' and 'auto'. Also, we tried a polynomial degree of 2, 3, and 4 to see the influence that the use of a polynomial kernel has on model complexity.

Running the grid search, the best parameters were found out to be $C = 10$, kernel = 'linear', gamma = 0.001, and polynomial degree = 2. With this setting, the best cross-validation accuracy achieved was 99.17%. The model was re-trained with the above-mentioned parameters and yielded 98.10% on a separate test set.

Logistics Regression

Then we tuned the Logistic Regression model by grid search to find the best hyperparameters. We tried different hyperparameters in combination, such as the regularization parameter C , solvers-which are usually algorithms that help to find the best fit-and the maximum number of iterations, a metric that describes how many times the algorithm will run to find that best solution.

Then, the grid was performed with values $C=0.01, 0.1, 1, 10, 100$, solver='newton-cg', 'lbfgs', 'sag', max_iter=100, 200, 300, 500. For the multi_class parameter, we use a different strategy depending on the solver. Thus, for those solvers that are able to handle multinomial classification, namely 'newton-cg', 'lbfgs', and 'sag', this was set to 'multinomial', which optimizes the probabilities of all classes at once. For the 'liblinear', which does not support multiclass classification, we used the 'ovr' strategy-training individual classifiers for each class.

The best parameters were $C = 100$, solver = 'newton-cg', max_iter = 100, and multi_class strategy = 'multinomial'. With these settings, the model performed very well in cross-validation, an accuracy of 98.93%, while after a retraining with the best parameters, we have got an accuracy of 97.86% on a different test set.

Random Forest

To tune the Random Forest model, we used RandomizedSearchCV, which explores a variety of combinations of parameters. It focused on a number of relevant parameters: `n_estimators`, which is the regulation of the number of trees in the forest; `max_depth`, which controls the greatest depth of each tree; `min_samples_split`, which regulates the minimum number of samples necessary to split a node; `min_samples_leaf`, regulating the minimum number of samples necessary to be at each leaf node; `max_features`, identifying the number of features to consider at every split; and `bootstrap`, which is a boolean value used for controlling whether to use bootstrapping or not, that is, sampling with replacement.

The range of trees we tried varied between 100 and 500. We have also used a wide range of maximum depths for the trees from unlimited depth to 40. The minimum number of samples required to split a node was varied between 2, 5, and 10, and minimum number of samples per leaf had values of 1, 2 and 4. Besides that, we tried two ways of deciding how many features to consider at each split: using either square root of total features ('sqrt') or base-2 logarithm ('log2'). We tried the models with and without bootstrapping, too. 300 trees, a maximum depth of 40, the minimum number of samples to split a node was 2, samples per leaf were also 2, 'sqrt' for the number of features, no bootstrapping. The best cross-validation accuracy achieved by the model was 95.60% for these settings. When tested on the separate test set, its performance improved further, yielding an accuracy of 97.38%.

Chapter 3: Result and Discussion

3.1: Training and Testing Results

3.1.1: Training Accuracy

	K-Nearest Neighbour	Support Vector Classifier	Logistic Regression	Random Forest
Accuracy =	0.8351	0.9488	0.9726	0.9565
Precision =	0.8361	0.9418	0.9684	0.9491
Recall =	0.8038	0.9423	0.9674	0.9501
f1 Score =	0.8153	0.9411	0.9676	0.9496

Figure 3.1: Before SMOTE – Training Scores

Before SMOTE, models generally show high accuracy but struggle with minority classes. For instance, kNN achieves 83.51% accuracy but has lower recall (80.38%), indicating difficulty in predicting 'low' and 'medium' classes. Similarly, SVC, logistic regression, and random forest models perform well overall but are biased towards the majority class, resulting in lower recall for minority classes.

	K-Nearest Neighbour	Support Vector Classifier	Logistic Regression	Random Forest
Accuracy =	0.9091	0.9740	0.9804	0.9740
Precision =	0.9108	0.9745	0.9810	0.9741
Recall =	0.9091	0.9740	0.9804	0.9740
f1 Score =	0.9094	0.9741	0.9805	0.9740

Figure 3.2: After SMOTE – Training Scores

After applying SMOTE, there is a notable improvement in all metrics. The kNN model's accuracy rises to 90.91%, with recall improving to 90.91%, reflecting better prediction of minority classes. The SVC, logistic regression, and random forest models also show significant gains, with accuracy and recall improving across the board. SMOTE effectively balances class predictions, leading to more robust and fair model performance.

3.1.2: Testing Accuracy

	K-Nearest Neighbour	Support Vector Classifier	Logistic Regression	Random Forest
Accuracy =	0.7881	0.9024	0.9429	0.9190
Precision =	0.7784	0.8947	0.9336	0.9099
Recall =	0.7577	0.8999	0.9339	0.9091
f1 Score =	0.7634	0.8939	0.9336	0.9094

Figure 3.3: Before SMOTE – Testing Scores

Before SMOTE, the models had high accuracy but struggled with predicting minority classes. For instance, the kNN model had 78.81% accuracy and a recall of 75.77%, showing difficulty with minority classes. Similarly, SVC, logistic regression, and random forest models performed well overall but had lower recall for minority classes, indicating a bias towards the majority class.

	K-Nearest Neighbour	Support Vector Classifier	Logistic Regression	Random Forest
Accuracy =	0.8441	0.9337	0.9498	0.9373
Precision =	0.8509	0.9387	0.9518	0.9371
Recall =	0.8441	0.9337	0.9498	0.9373
f1 Score =	0.8455	0.9345	0.9500	0.9372

Figure 3.4: After SMOTE – Testing Scores

After applying SMOTE, all models improved significantly. The kNN model's accuracy increased to 84.41% with recall rising to 84.41%. The SVC model's accuracy went up to 93.37%, and recall improved to 93.37%. Logistic regression saw accuracy rise to 94.98% and recall to 94.98%, while the random forest model's accuracy and recall both increased to 93.73%. SMOTE helped balance the predictions, leading to better performance for all models, especially in predicting minority classes.

3.2: In-Depth Analysis of Prediction Performance

3.2.1: Confusion Matrix on Training Set

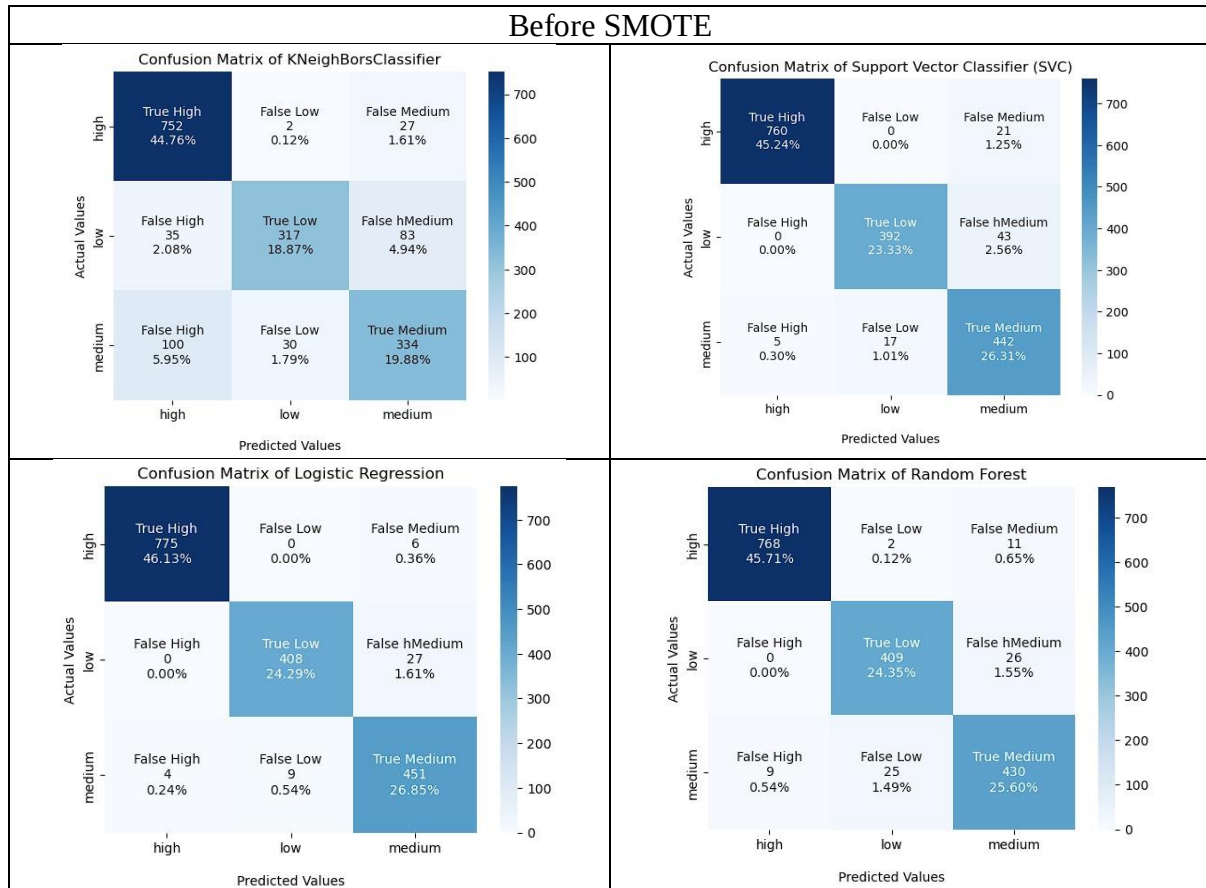


Figure 3.5: Before SMOTE – Confusion Matrix of Each Models

In the imbalanced dataset, all models show a strong bias toward the majority class (high), with most instances of this class being correctly classified. However, the minority classes (low and medium) experience significant misclassification. For example, in the kNN model, the high class is well-predicted, but the medium and low classes are often misclassified, with high confusion between these categories. This pattern is consistent across all models, where the imbalance causes poorer performance in identifying minority classes.

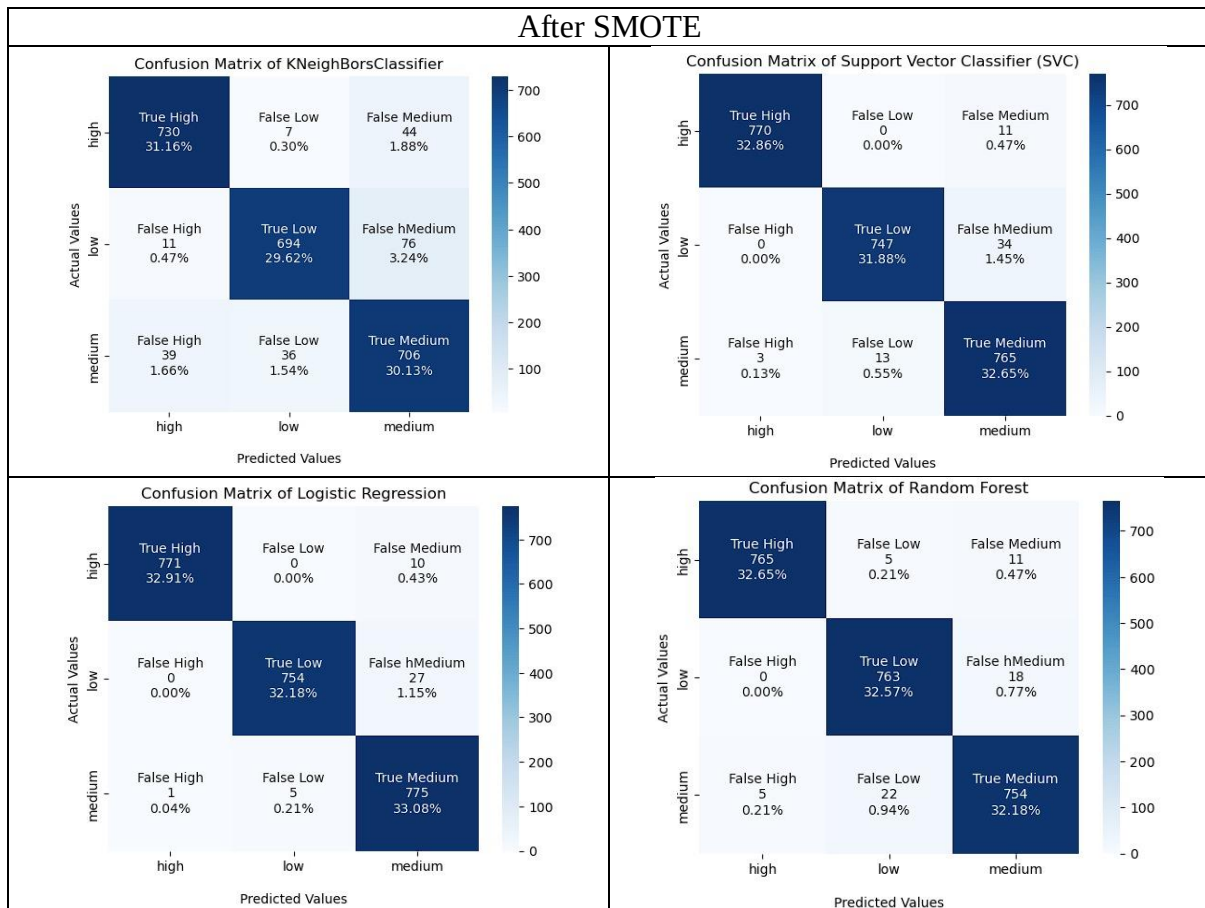


Figure 3.6: After SMOTE – Confusion Matrix of Each Models

After applying SMOTE, there is an obvious improvement in the classification of minority classes (low and medium) across all models. For instance, the kNN model shows significantly better predictions for the medium and low classes, while still maintaining accuracy for the high class. Other models, like SVC and logistic regression, also exhibit similar improvements, with far fewer misclassifications for minority classes. SMOTE effectively balances the dataset, leading to more equitable and accurate classification across all classes.

3.2.2: ROC and PR Curves on Training Set (before SMOTE)

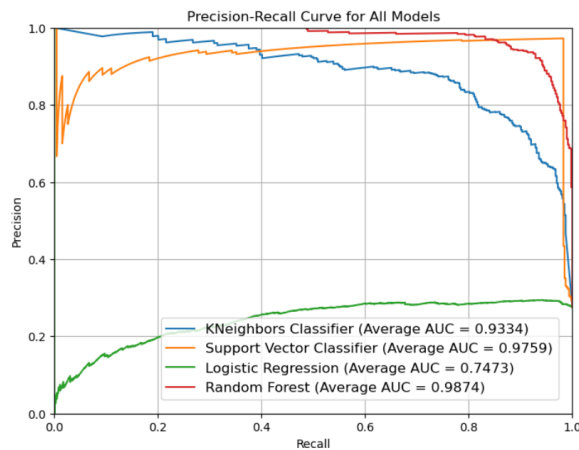


Figure 3.7: P-R Curve

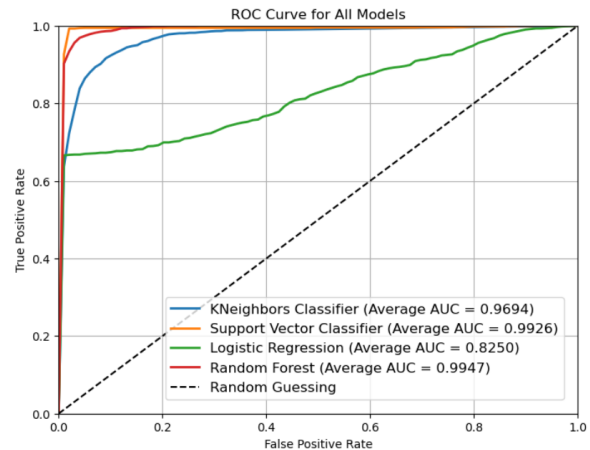


Figure 3.8: ROC Curve

Before applying SMOTE, the SVC model stood out as the best-performing model. With an average AUC of 0.9759 in the Precision-Recall (PR) curve and 0.9926 in the ROC curve, it effectively handled the class imbalance in the dataset. The high AUC scores indicate that SVC achieved a strong balance between precision and recall, making it better at correctly identifying both the majority and minority classes. Its performance in distinguishing true positives from false positives was superior to other models, making it the most reliable classifier before SMOTE.

In comparison, the Random Forest model showed excellent performance in the PR curve (AUC = 0.9874) but had a lower AUC in the ROC curve (0.9047), indicating that while it performed well in precision and recall, it was slightly less effective in handling false positives. The kNN model performed moderately well in both PR (AUC = 0.9334) and ROC (AUC = 0.9694), and Logistic Regression lagged behind with the lowest AUC scores (PR AUC = 0.7473, ROC AUC = 0.8250), struggling with class imbalance.

3.2.3: ROC and PR Curves on Training Set (after SMOTE)

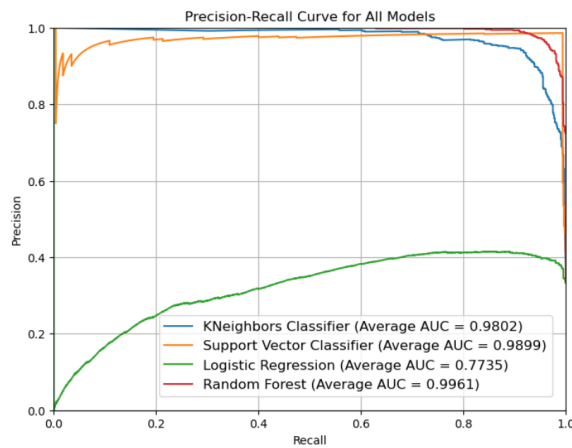


Figure 3.9: P-R Curve

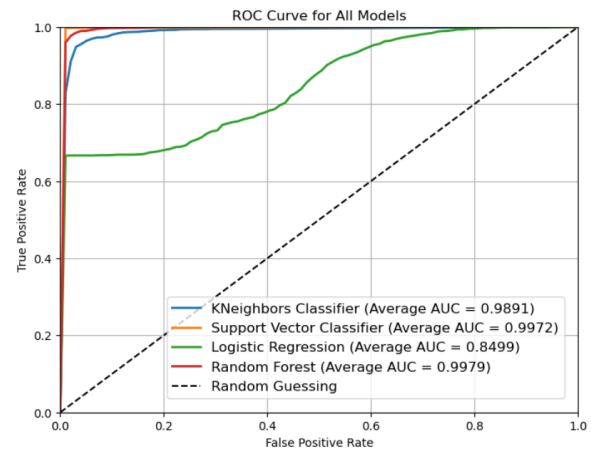


Figure 3.10: ROC Curve

After SMOTE, Random Forest model became the top performer, achieving the highest AUC scores across both metrics. Its PR curve AUC of 0.9961 and ROC curve AUC of 0.9979 indicate that it provided the best overall classification performance. The Random Forest model excelled at distinguishing between classes and maintaining a balance between precision and recall, proving highly effective after the data was balanced with SMOTE.

The SVC model followed closely with an AUC of 0.9899 in the PR curve and 0.9972 in the ROC curve, maintaining strong performance but slightly behind Random Forest. The kNN model also improved significantly with SMOTE, achieving an AUC of 0.9802 in the PR curve and 0.9891 in the ROC curve. However, the Logistic Regression model, despite improving from its pre-SMOTE performance, remained the weakest model (PR AUC = 0.7735, ROC AUC = 0.8499), indicating it still struggled to fully capture the balance between positive and negative instances in the now balanced dataset.

3.2.4: Models Performance between Training Set and Test Set (after SMOTE)

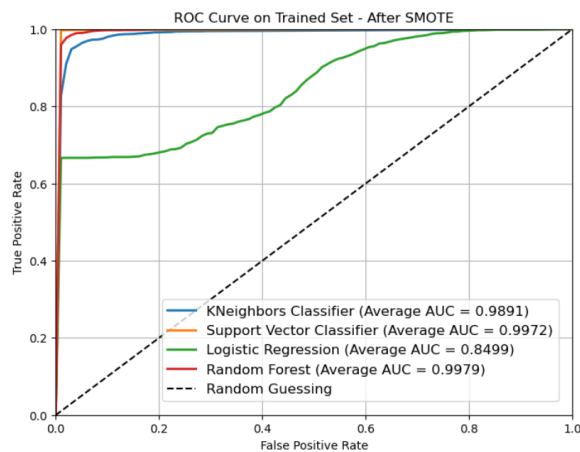


Figure 3.11: Trained Set

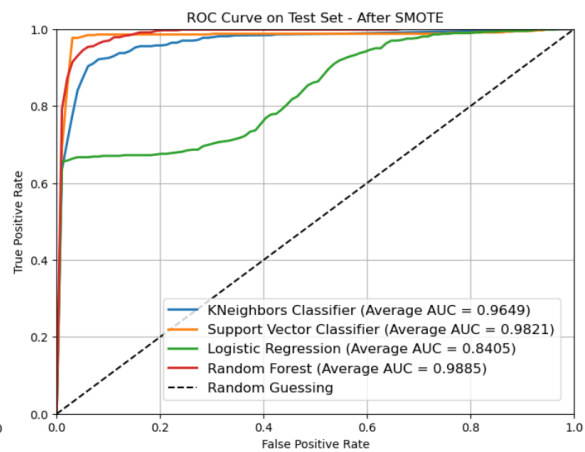


Figure 3.12: Test Set

By observing the figures above, we can see that there are no substantial gaps between the training and test sets for any of the models, although some models exhibit minor signs of overfitting. The SVC and Random Forest models demonstrate the best balance between training and test performance, indicating they are well-suited to the data with limited overfitting. While Logistic Regression shows consistency between training and test results, it underperforms compared to the other models.

3.3: Strengths and Weaknesses of Models

K-Nearest Neighbors Classifier (k-NN)

Strengths:

- **Easy to Understand**
kNN is simple and straightforward, making it easy to grasp and implement.
- **No Training Required**
it doesn't need a training phase, so updating with new data is simple.
- **Flexible**
works with different types of data and distance metrics.

Weaknesses:

- **Slow Predictions**
can be slow when making predictions, especially with large datasets.
- **Struggles with High Dimensions**
performance drops with many features (dimensions).
- **Sensitive to Noise**
can be affected by noisy data or outliers.
- **Needs Careful Tuning**
performance depends on choosing the right number of neighbors (k) and distance metric.

Support Vector Classifier (SVC)

Strengths:

- **Good with High Dimensions**
works well with lots of features and clear class separations.
- **Prevents Overfitting**
regularization helps avoid overfitting, especially in high-dimensional data.
- **Handles Non-linear Boundaries**
can model complex relationships using different kernels.

Weaknesses:

- **Computationally Heavy**
training and predictions can be slow, especially with large datasets.
- **Hard to Tune**
needs careful tuning of parameters like C, kernel, and gamma.
- **Less Transparent**
the model's decision-making is less clear compared to simpler models.
- **Scalability Issues**
can be challenging with very large datasets.

Logistic Regression

Strengths:

- **Simple and Clear**
easy to understand and interpret, showing clear relationships between features and outcomes.
- **Fast and Efficient**
quick to train and make predictions, even with large datasets.
- **Provides Probabilities**
gives probabilities for predictions, useful in many applications.

Weaknesses:

- **Assumes Linearity**
assumes a straight-line relationship, which may not fit complex data well.
- **Needs Feature Scaling**
works best when features are scaled properly.
- **Doesn't Capture Interactions**
struggles with modeling interactions between features.
- **Can Overfit**
may overfit with many features if not regularized.

Random Forest

Strengths:

- **Robust and Reliable**
handles noisy data well and reduces overfitting by averaging multiple decision trees.
- **Versatile**
works with different types of data and can model complex relationships.
- **Shows Feature Importance**
helps identify which features are most important.

Weaknesses:

- **Computationally Demanding**
training and prediction can be slow, especially with many trees.
- **Less Interpretable**
harder to understand how decisions are made compared to single decision trees.
- **Slower Predictions**
can be slow when making predictions if there are many trees.
- **Requires Tuning**
needs careful tuning of parameters like the number of trees and tree depth.

3.4: Features Importance

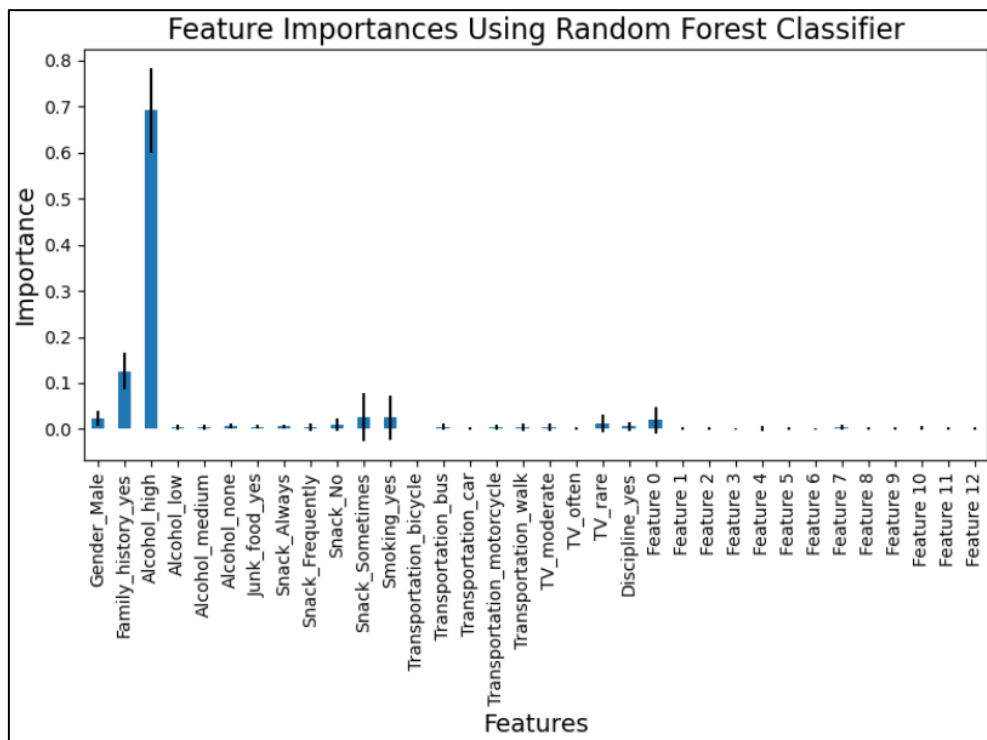


Figure 3.13: The bar chart of all possible input values

Alcohol_high	0.691606
Family_history_yes	0.125312
Snack_Sometimes	0.026198
Smoking_yes	0.024678
Gender_Male	0.023046
Feature 0	0.019152
TV_rare	0.011192
Snack_No	0.009813
Snack_Always	0.005734
Alcohol_none	0.005731
Discipline_yes	0.005341
Transportation_bus	0.004696
Snack_Frequently	0.004669
Junk_food_yes	0.004232
Feature 7	0.003531
Transportation_walk	0.003224
Transportation_motorcycle	0.003197
Alcohol_medium	0.003169
TV_moderate	0.003140
Alcohol_low	0.002896
Feature 10	0.002212
Feature 4	0.002130
Feature 2	0.001793
Feature 12	0.001765
Feature 1	0.001693
Feature 11	0.001639
Transportation_car	0.001637
TV_often	0.001589
Feature 9	0.001449
Feature 8	0.001309
Feature 5	0.001134
Feature 6	0.000633
Feature 3	0.000461
Transportation_bicycle	0.000000

Figure 3.14: The rankings of all possible input values from highest to lowest

The feature importance analysis from the Random Forest model reveals key insights into the predictors influencing the target variable. The most influential feature is Alcohol_high, with a substantial importance score of 0.691606, indicating that high levels of alcohol consumption have a significant impact on the model's predictions. Following closely is Family_history_yes, with a score of 0.125312, suggesting that having a family history of cardiovascular issues is also a critical factor.

Moderately important features include Snack_Sometimes, Smoking_yes, and Gender_Male, reflecting their moderate influence on the target variable. However, their lower importance compared to the top features suggests they play a less pivotal role in the model's predictions. Less important features, such as numerical ones labeled as Feature 0, Feature 7, and others, contribute less to the model's performance. This indicates that these features might either be less relevant or less informative in the context of the target variable.

Overall, the analysis underscores the significance of categorical features related to lifestyle and health history, highlighting their strong influence on the model's predictions. This information can guide further investigations into feature engineering and model refinement.

Chapter 4: Conclusion

In this project, we built and evaluated several machine learning models—KNN, SVC, Logistic Regression, and Random Forest—to predict cardiovascular risk. Initially, the models had trouble predicting the minority classes due to imbalanced data, but using SMOTE improved their accuracy, particularly in identifying those underrepresented cases. Random Forest turned out to be the best model, achieving the highest scores in important metrics like AUC, and performed well in balancing precision and recall.

The analysis also highlighted that factors like alcohol consumption and family history play the most significant roles in predicting cardiovascular risk. This shows how both lifestyle habits and genetic factors contribute to cardiovascular health. While each model had its pros and cons—KNN was easy to interpret, SVC handled complex data well, and Logistic Regression was straightforward—Random Forest was chosen for its balance of accuracy and reliability.

In the future, incorporating more data and exploring advanced methods could make these models even more useful, especially in clinical settings where they can help assess and prevent cardiovascular risks.